



Lecture 2: Installing and Configuring Python

Everything you need on your computer to get started in this class!

Contents

1. [Environment overview](#)
2. [What if I already have Python?](#)
3. [A Primer on Environments](#)
4. [Installing and configuring Python with Miniconda](#)
5. [Installing and configuring VSCode](#)

Programming Environment Overview

Where you write

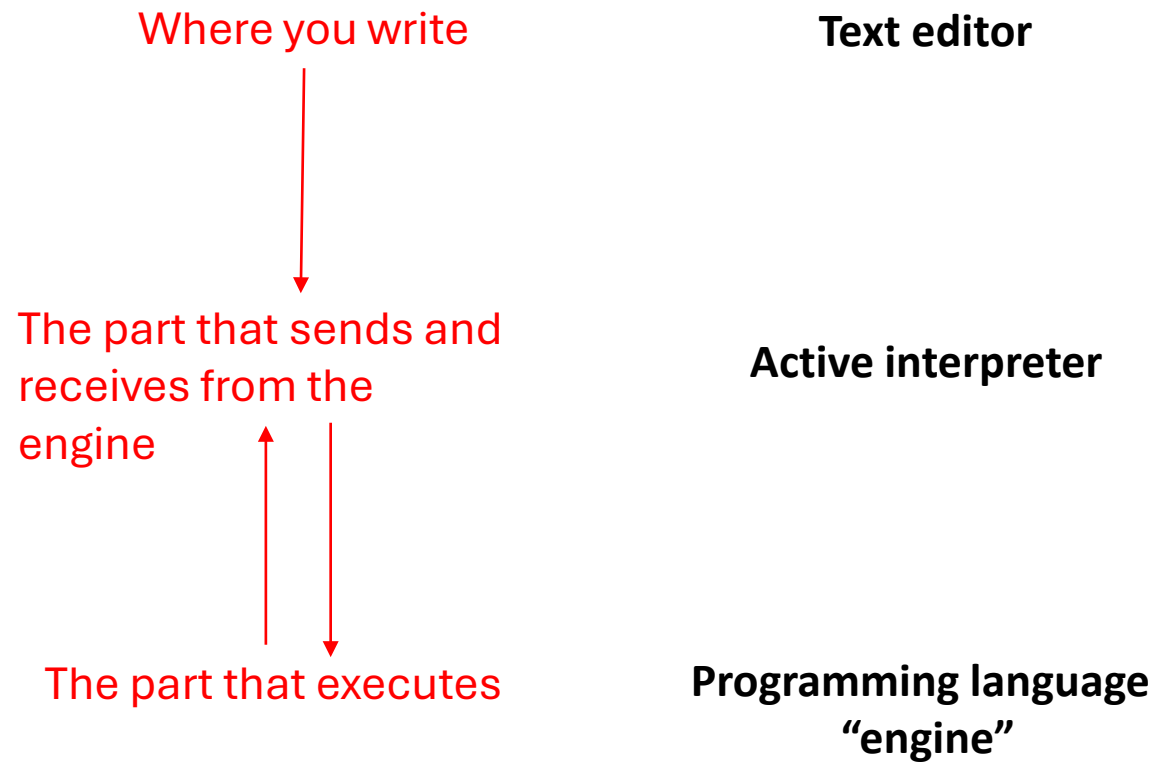
Text editor

Active interpreter

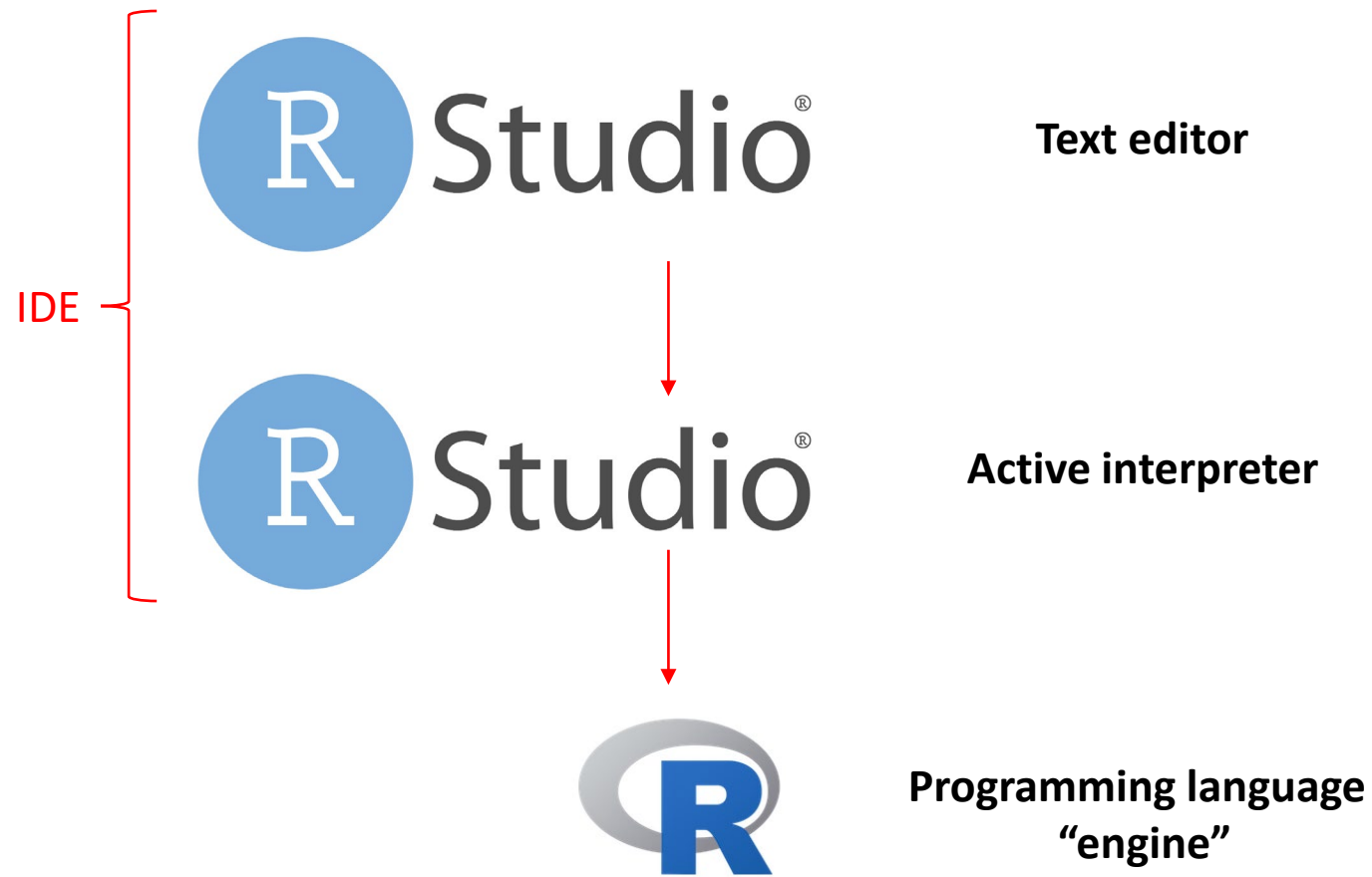
The part that executes

**Programming language
“engine”**

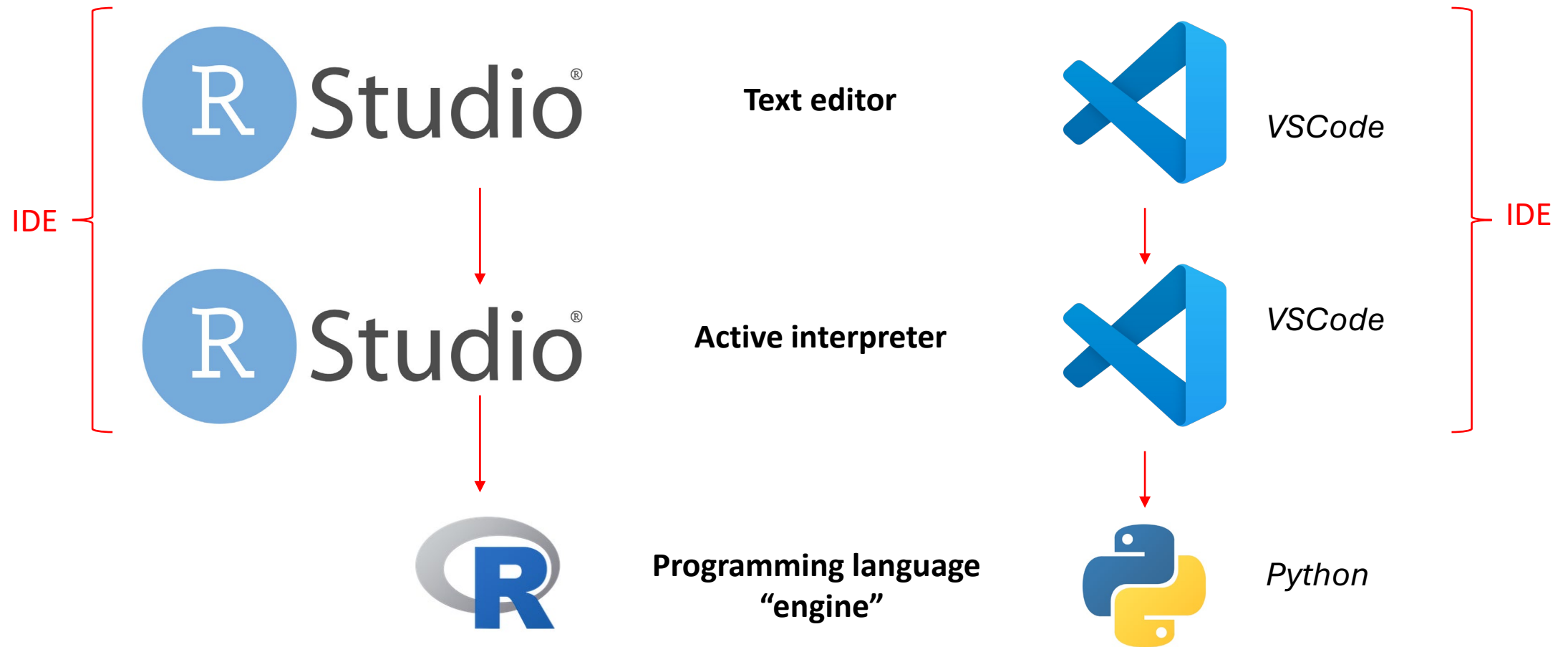
Programming Environment Overview



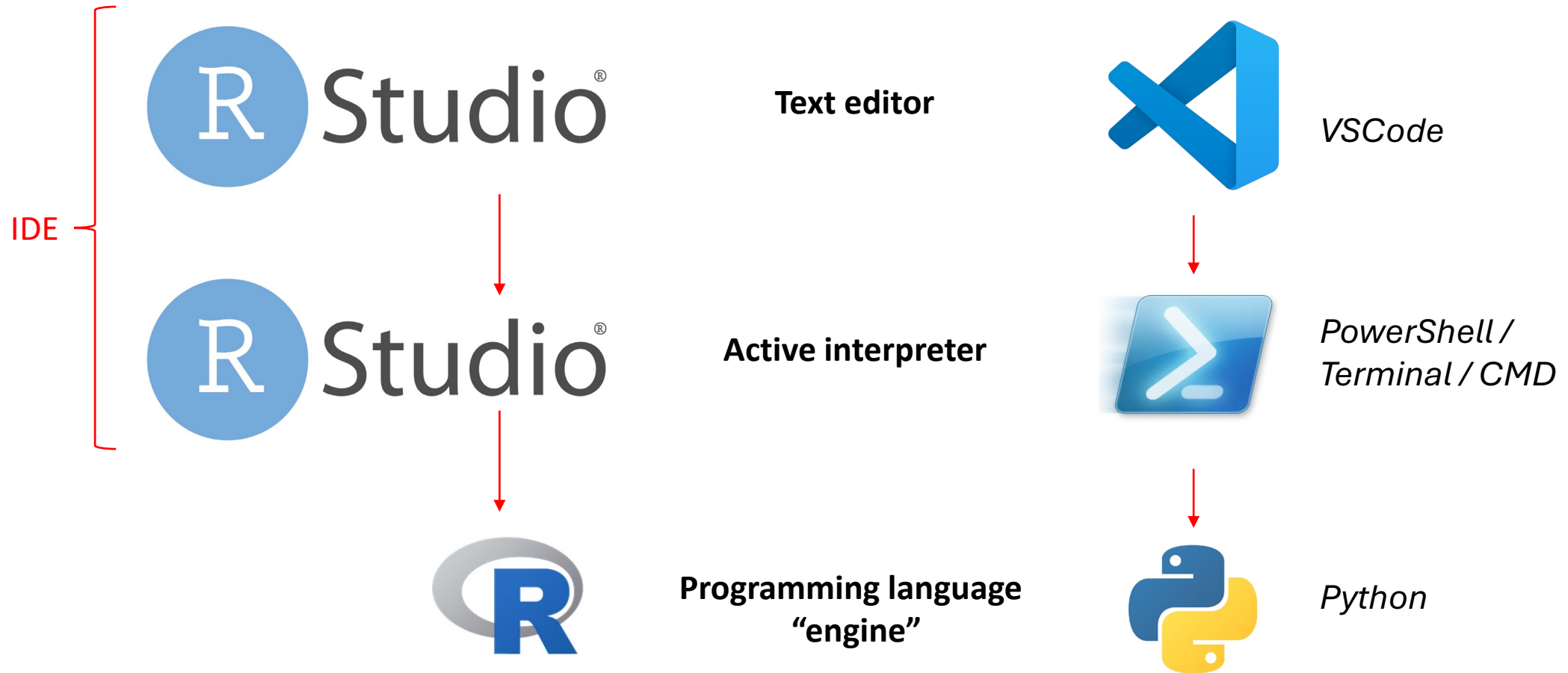
The R Environment



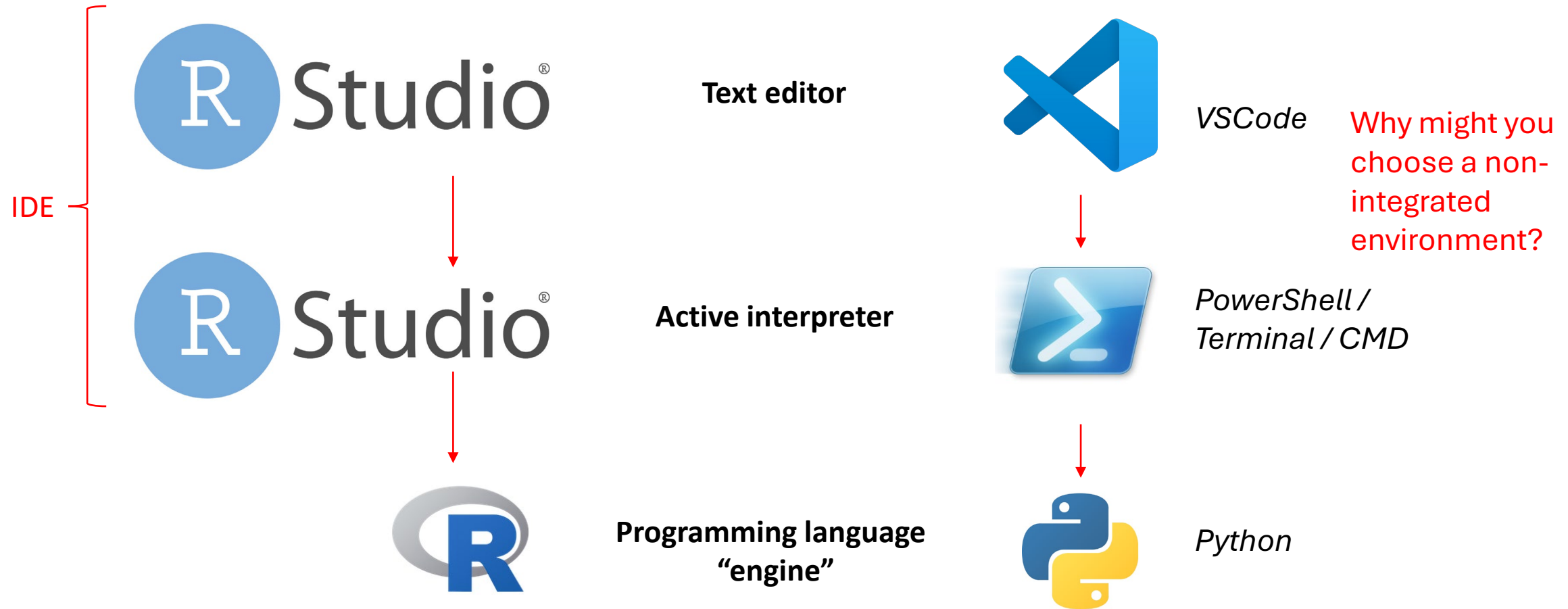
The Python Environment



Alternative Python Environment



Alternative Python Environment



What if I Already Have Python?

- The choice of development environment is completely **up to you**
 - It does not change anything about the final product
- That said, we strongly suggest you follow our recommendations
- If you prefer not to, it may be harder for us to support you
- You can also always change your development environment later!

A Primer on Environments

Imagine this scenario:

1. You start a big research project in Python using **Pandas version 1** & **Matplotlib version 2**

A Primer on Environments

Imagine this scenario:

1. You start a big research project in Python using **Pandas version 1** & **Matplotlib version 2**
2. You then get a new job! Congratulations! The project they put you on uses **Pandas version 2** & **Matplotlib version 3**

A Primer on Environments

Imagine this scenario:

1. You start a big research project in Python using **Pandas version 1** & **Matplotlib version 2**
2. You then get a new job! Congratulations! The project they put you on uses **Pandas version 2** & **Matplotlib version 3**
3. You update your versions of Pandas and Matplotlib
4. Now your first research project no longer works!

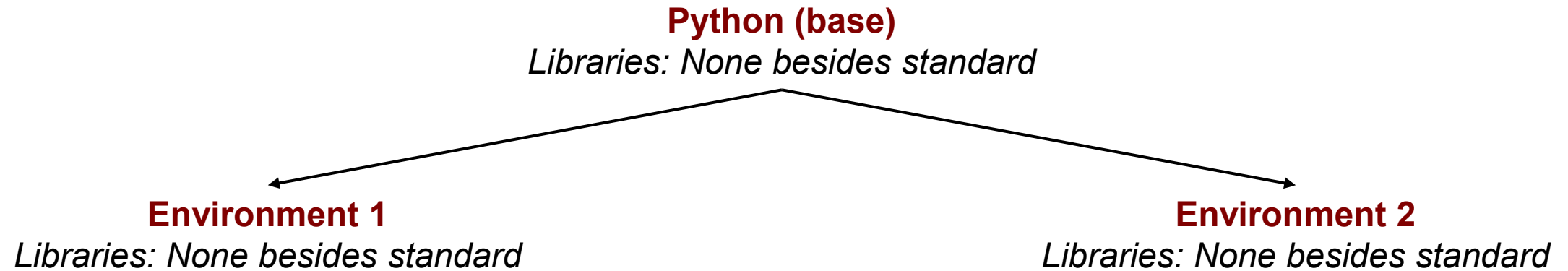
What can we do about this???

A Primer on Environments

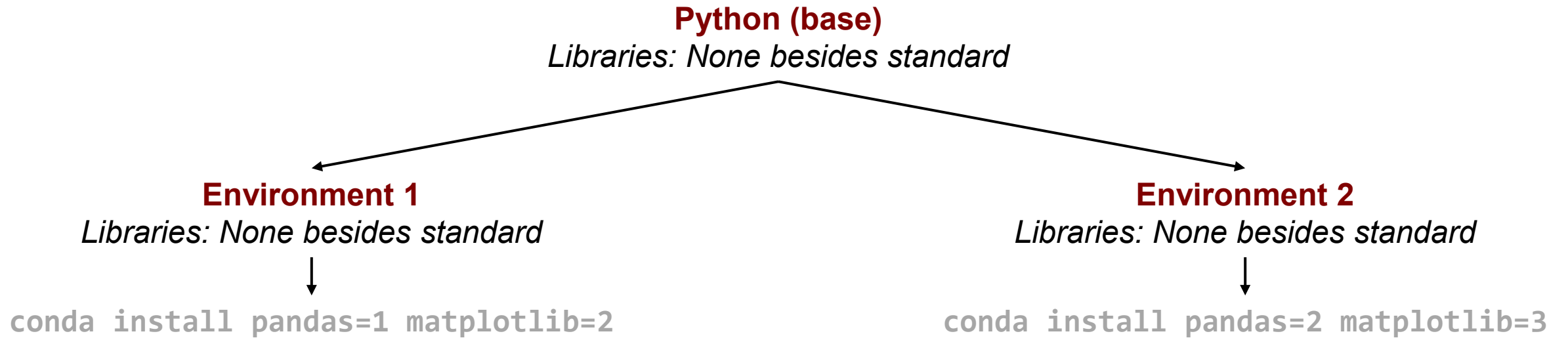
Python (base)

Libraries: None besides standard

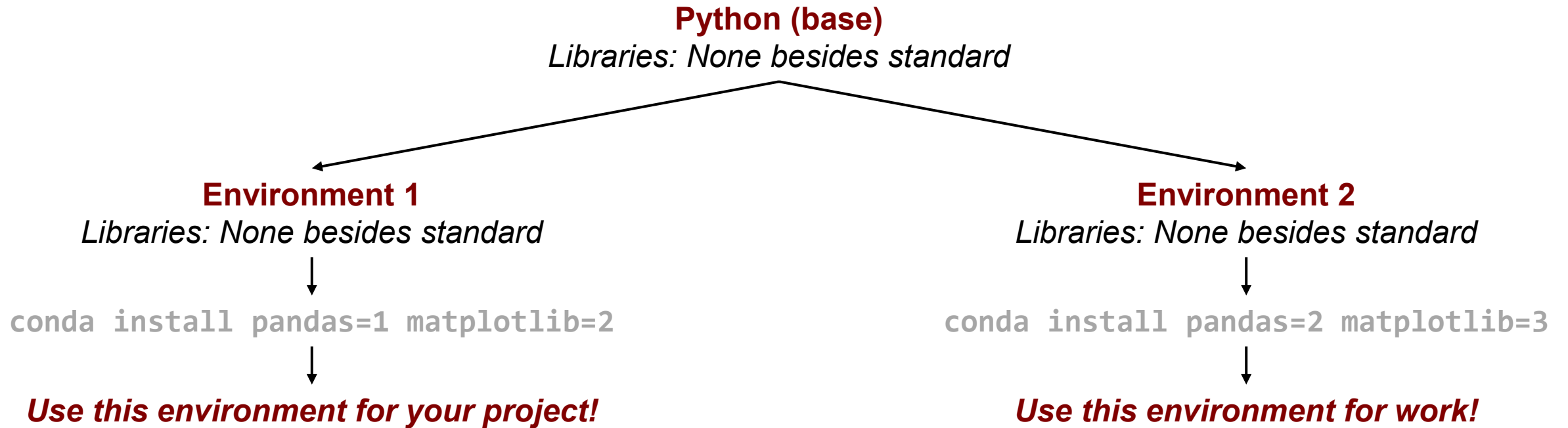
A Primer on Environments



A Primer on Environments



A Primer on Environments



A Primer on Environments

In this class:

- We will create one environment and use it all the time
- Our intention is that it also serves you through DAP2 and ML!

A Primer on Environments

In this class:

- We will create one environment and use it all the time
- Our intention is that it also serves you through DAP2 and ML!

Also good for:

- Preventing conflicts between many libraries that may not work well together
- Keeping your Python from becoming “bloated” with hundreds of libraries
- Sharing projects with others (Coworkers? Coauthors? Public users?)

1. Install Miniconda

<https://www.anaconda.com/download/success>

- Do not install Anaconda!
 - It is at the top!
 - Scroll past that & select Miniconda
- Select Windows, Mac, or Linux
- Download & run the installer



Miniconda Installers



Windows

Python 3.12

⬇ 64-Bit Graphical Installer



Mac

Python 3.12

⬇ 64-Bit (Apple silicon)
Graphical Installer



Linux

Python 3.12

⬇ 64-Bit (x86) Installer

Select your OS

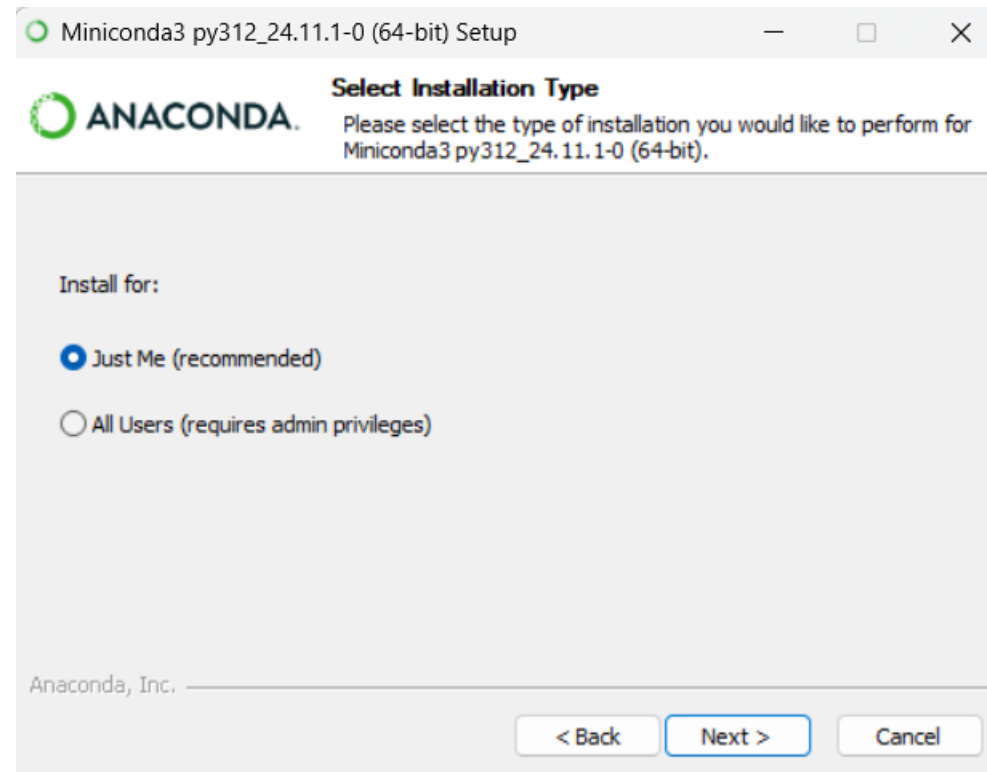
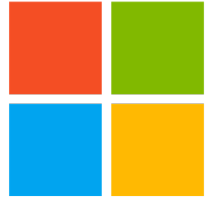
[Install Miniconda – Windows](#)



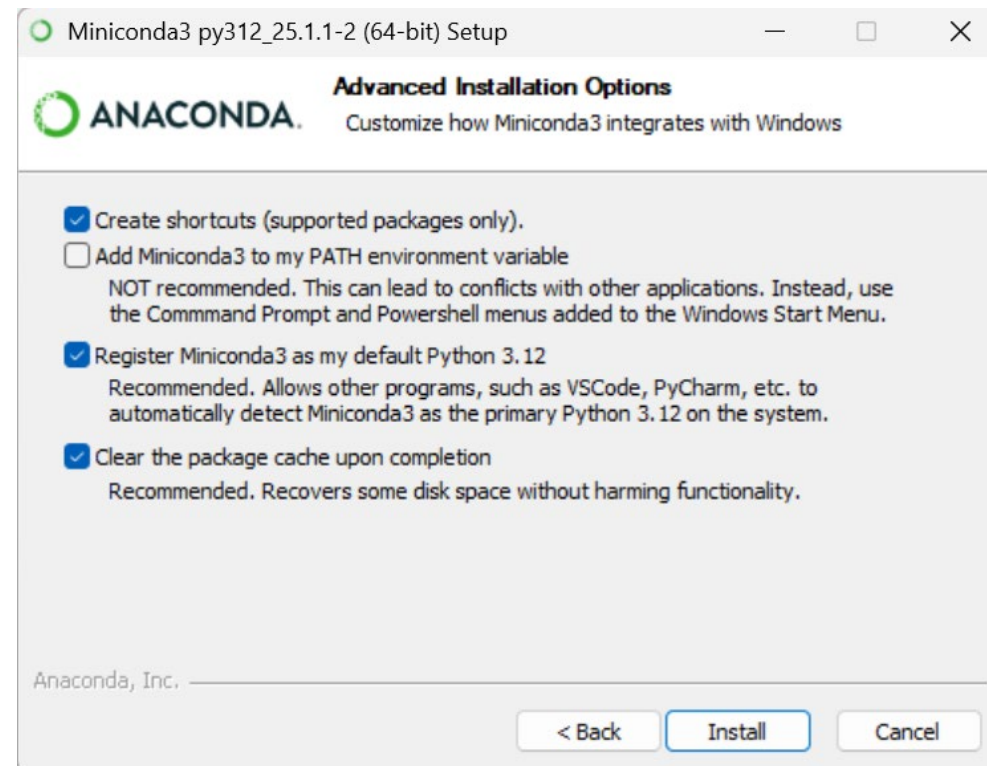
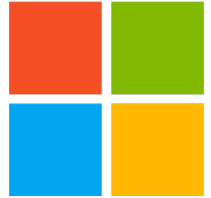
[Install Miniconda - Mac](#)

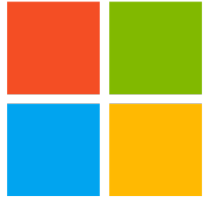


1. Install Miniconda - Windows



1. Install Miniconda





2. Create environment

First launch “Anaconda Prompt” from your Window menu, then:

“y” to proceed

```
Anaconda Prompt - conda cr X + v
(base) C:\Users\JTest>conda create --name DAP
Channels:
- defaults
Platform: win-64
Collecting package metadata (repodata.json): done
Solving environment: done

==> WARNING: A newer version of conda exists. <==
current version: 25.1.1
latest version: 25.3.0

Please update conda by running

$ conda update -n base -c defaults conda

## Package Plan ##

environment location: C:\Users\JTest\miniconda3\envs\DAP

Proceed ([y]/n)?
```



3. Install DAP 1&2 Libraries

1. Type: `conda activate DAP`
2. Download: The file `requirements.txt` from Canvas
3. Navigate: To the folder with this file
or specify the full path as below
4. Type:

```
conda install --yes --file requirements.txt -c defaults -c conda-forge
```

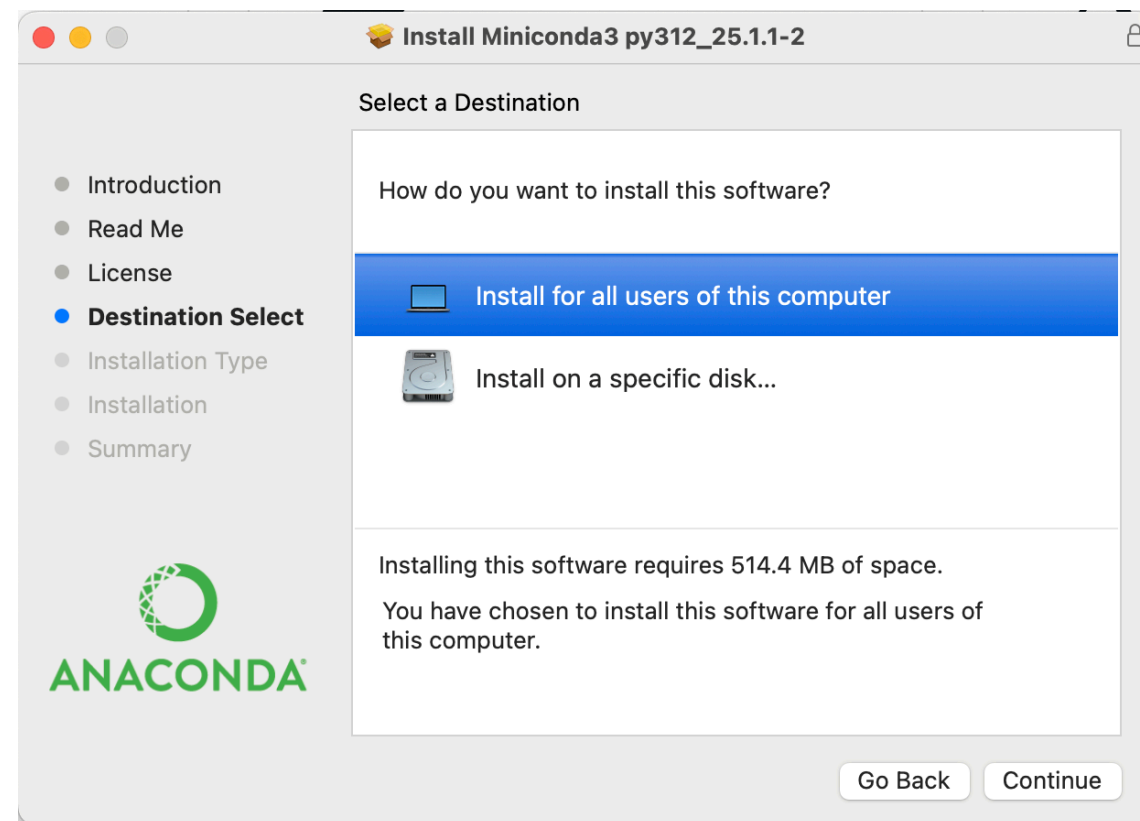
```
(DAP) C:\Users\JTest>conda install --yes --file c:\users\jtest\downloads\requirements.txt -c default -c conda-forge
Channels:
- default
- conda-forge
- defaults
Platform: win-64
Collecting package metadata (repodata.json): done
Solving environment: done
```

Note: “Solving environment”
sometimes takes a while,
possibly even 10-20 minutes

Return to OS-agnostic slides

Don't Run Code From the Internet that You Don't Understand!

1. Install Miniconda - Mac





2. Create environment

First launch Terminal,
then:

```
tempuser — python ◀ conda create --name DAP — 80x39
Last login: Thu Feb 20 15:29:26 on console
[(base) tempuser@HPP-LNR-034 ~ % conda create --name DAP]
Channels:
  - defaults
Platform: osx-64
Collecting package metadata (repodata.json): done
Solving environment: done

==> WARNING: A newer version of conda exists. <==
  current version: 25.1.1
  latest version: 25.3.0

Please update conda by running

  $ conda update -n base -c defaults conda

## Package Plan ##

  environment location: /opt/miniconda3/envs/DAP

Proceed ([y]/n)? █
```

“y” to proceed



3. Install DAP 1&2 Libraries

1. In the same Terminal window, type `conda activate DAP`
2. Download: The file `requirements.txt` from Canvas
3. Navigate: To the folder with this file
4. Type:

```
conda install --file requirements.txt --channel defaults --channel conda-forge
```

```
1. [(base) tempuser@HPP-LNR-034 ~ % conda activate DAP ]
[(DAP) tempuser@HPP-LNR-034 ~ % cd downloads 3. ]
4. [(DAP) tempuser@HPP-LNR-034 downloads % conda install --yes --file requirements.t]
xt -c defaults -c conda-forge
Channels:
  - defaults
  - conda-forge
Platform: osx-64
Collecting package metadata (repodata.json): done
Solving environment: \ █
```

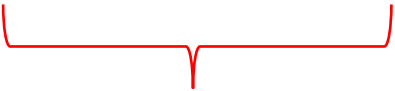
Note: “Solving environment”
sometimes takes a while,
possibly even 10-20 minutes

Return to OS-agnostic slides

Don't Run Code From the Internet that You Don't Understand!

Don't Run Code From the Internet that You Don't Understand!

```
conda install --yes --file requirements.txt -c defaults -c conda-forge
```



Tell the
Anaconda
package
manager that we
want to install

Don't Run Code From the Internet that You Don't Understand!

```
conda install --yes --file requirements.txt -c defaults -c conda-forge
```



Tell the
Anaconda
package
manager that we
want to install

Say “yes” to
any prompts
automatically

Don't Run Code From the Internet that You Don't Understand!

```
conda install --yes --file requirements.txt -c defaults -c conda-forge
```

A diagram with two red brackets. The first bracket is positioned under the words 'conda install' in the command line above. The second bracket is positioned under the words '--file requirements.txt' in the same command line.

Tell the
Anaconda
package
manager that we
want to install

Option to install libraries from
a list in this text file. We could
also type a single package
name here, for example
“pandas”.

Don't Run Code From the Internet that You Don't Understand!

```
conda install --yes --file requirements.txt -c defaults -c conda-forge
```

Tell the
Anaconda
package
manager that we
want to install

Option to install libraries from
a list in this text file. We could
also type a single package
name here, for example
“pandas”.

First look in
the default
package
management
location.

Don't Run Code From the Internet that You Don't Understand!

```
conda install --yes --file requirements.txt -c defaults -c conda-forge
```



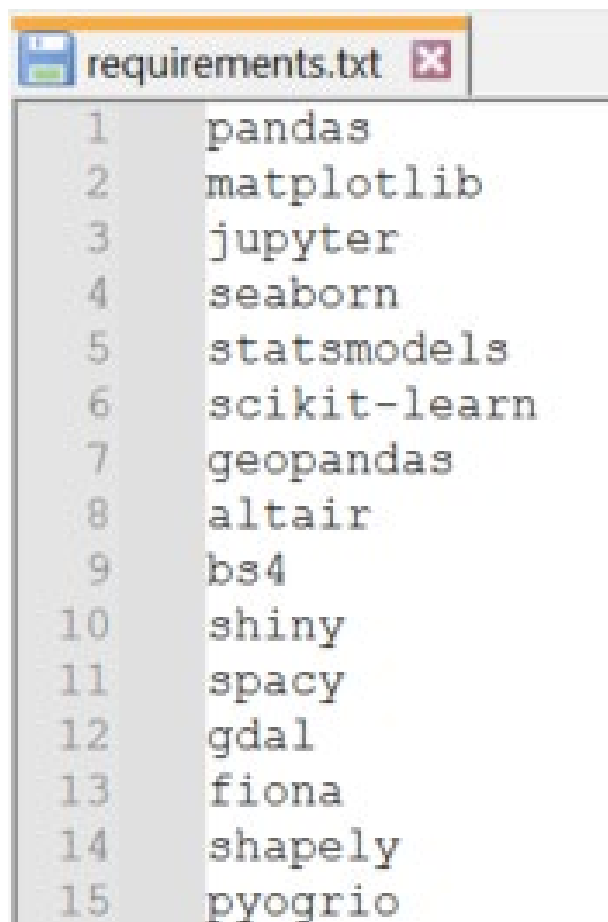
Tell the Anaconda package manager that we want to install

Option to install libraries from a list in this text file. We could also type a single package name here, for example “pandas”.

First look in the default package management location.

If it isn't in default, look in this second repository that is broader than the first.

What Did I Just Install?

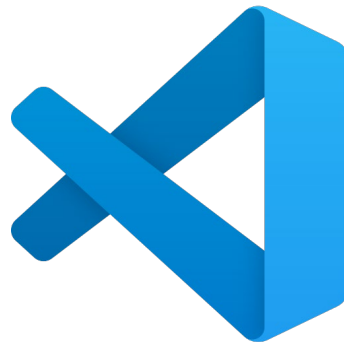


A screenshot of a text editor window titled 'requirements.txt'. The window displays a list of 15 Python packages, each preceded by a line number from 1 to 15. The packages are: pandas, matplotlib, jupyter, seaborn, statsmodels, scikit-learn, geopandas, altair, bs4, shiny, spacy, gdal, fiona, shapely, and pyogrio.

Line Number	Package Name
1	pandas
2	matplotlib
3	jupyter
4	seaborn
5	statsmodels
6	scikit-learn
7	geopandas
8	altair
9	bs4
10	shiny
11	spacy
12	gdal
13	fiona
14	shapely
15	pyogrio

4. VSCode

- The Python development environment is not unified the way R is around RStudio
- But VSCode is the closest thing to that for programming in general



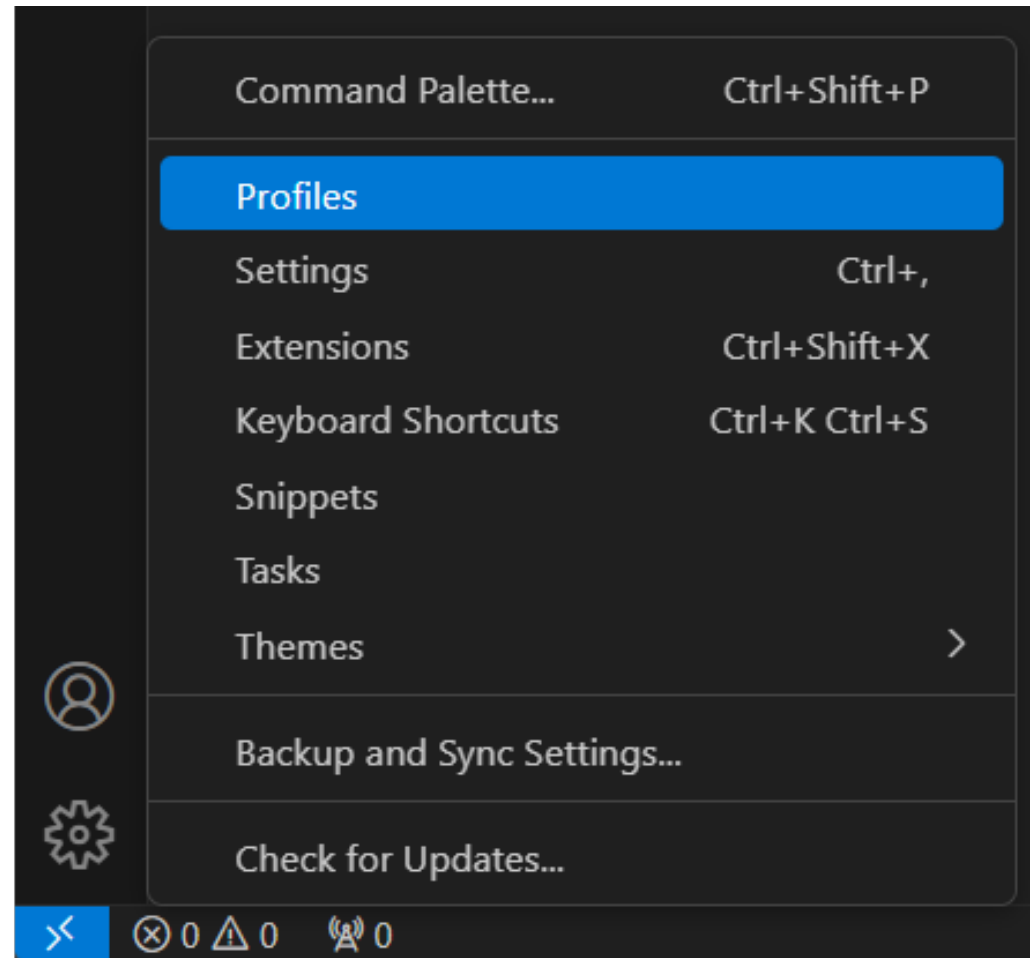
Download VSCode

<https://code.visualstudio.com/docs>

- Download and run the installer

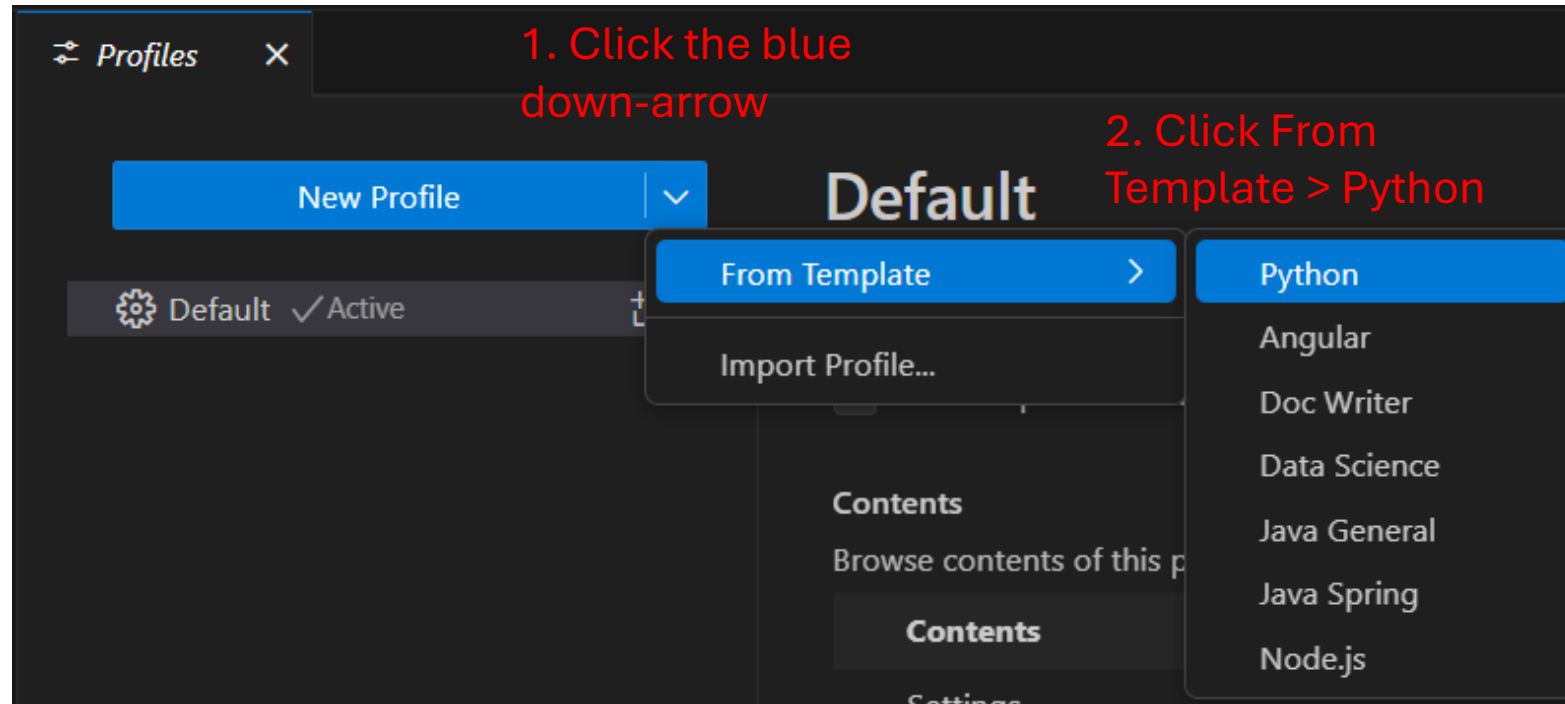
Setup a Profile

1. Click the gear in the lower-left corner



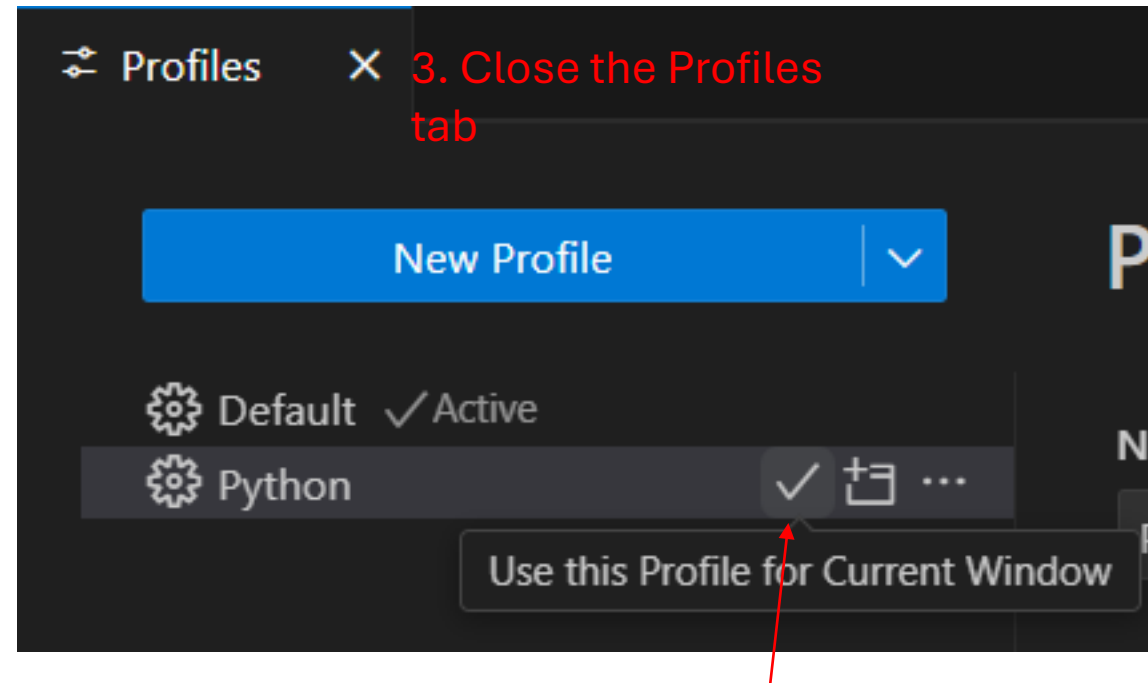
2. Click Profiles

Setup a Profile



Setup a Profile

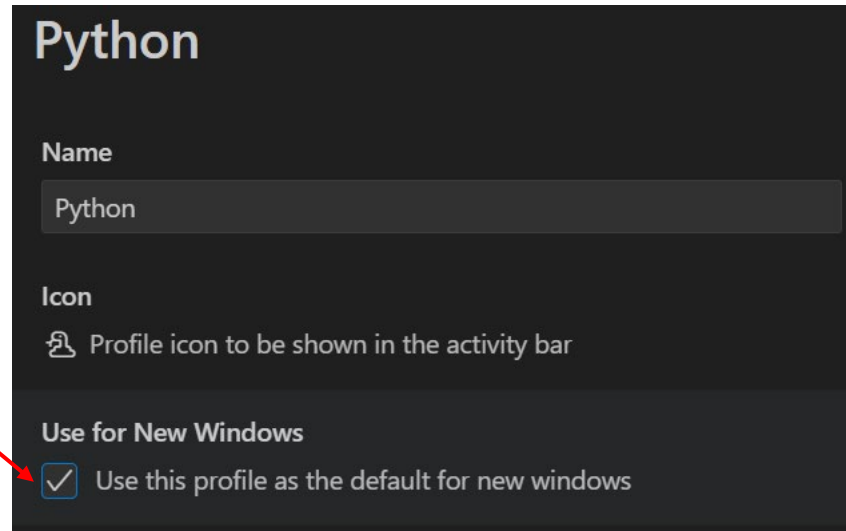
1. Click the blue “Create” button that appears at the bottom. This step will take a moment!



2. Click the check next to “Python” to activate this profile.

Setup a Profile

Check here so this
profile loads at
startup



Python

Name

Python

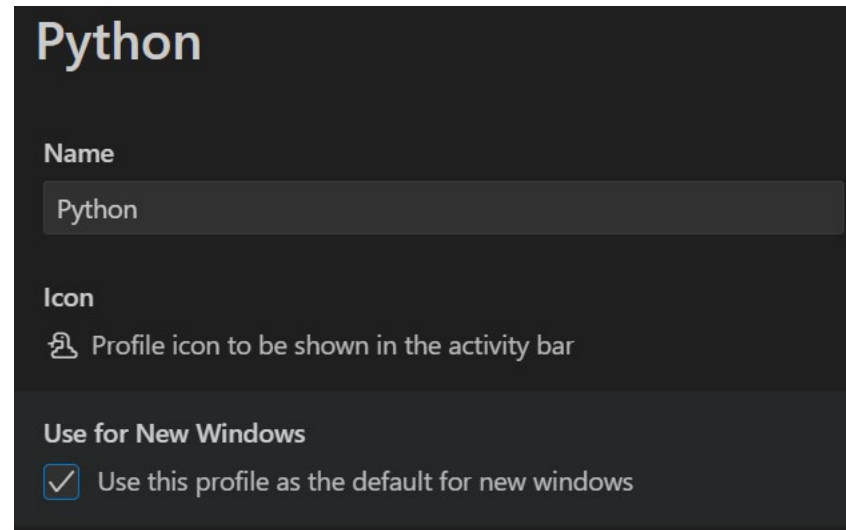
Icon

 Profile icon to be shown in the activity bar

Use for New Windows

☒ Use this profile as the default for new windows

Setup a Profile

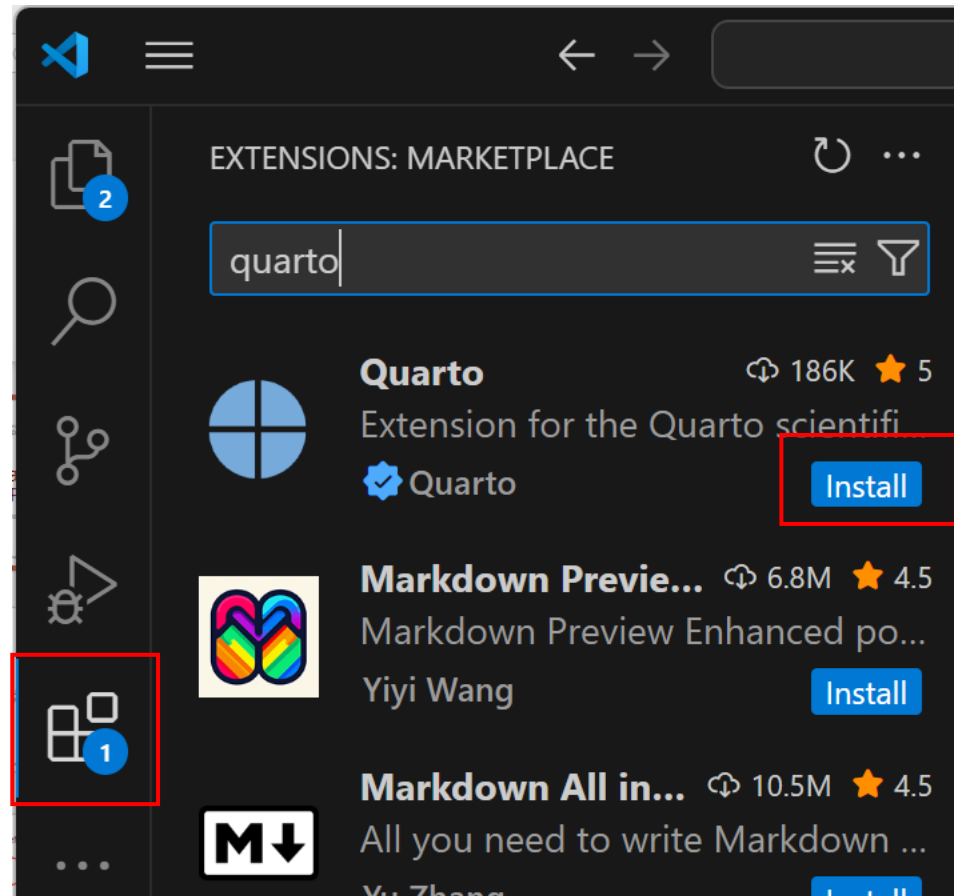


The screenshot shows a dark-themed window titled "Python". It contains three sections: "Name" with a text input field containing "Python"; "Icon" with a file icon and the text "Profile icon to be shown in the activity bar"; and "Use for New Windows" with a checked checkbox and the text "Use this profile as the default for new windows".

Finally, close the Profiles window

Install Quarto

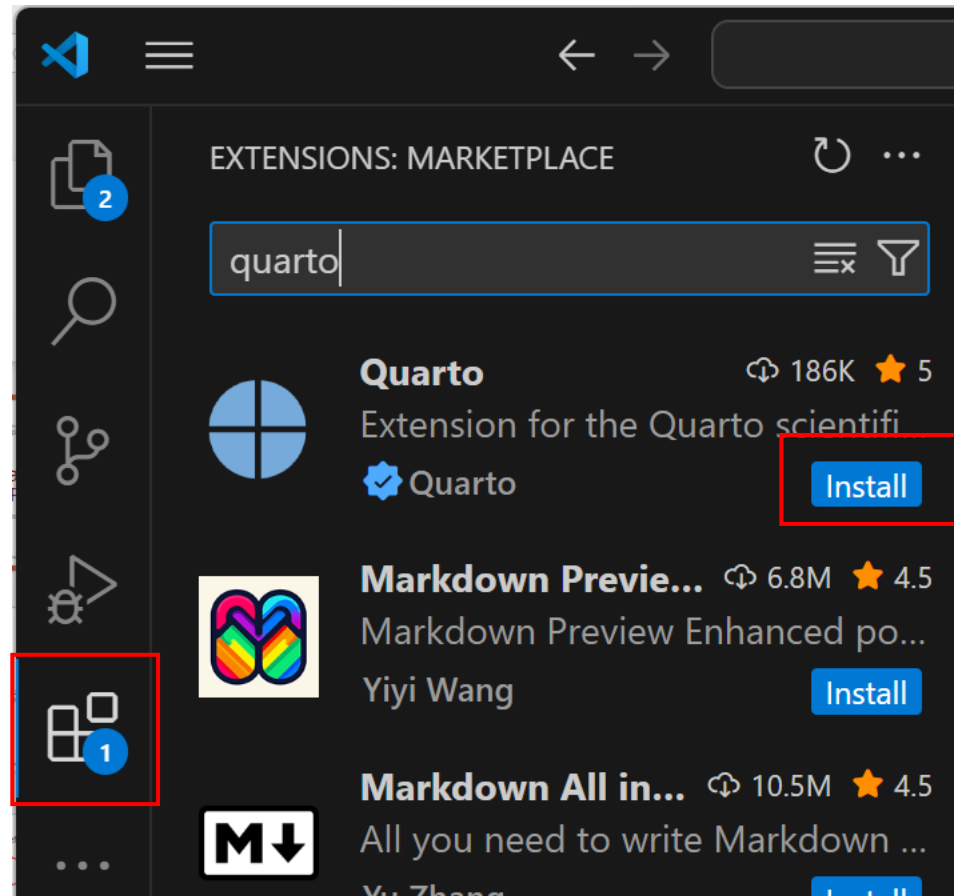
1. Click Extensions
on the VSCode side
bar



2. Click Install

Install Quarto

1. Click Extensions
on the VSCode side
bar



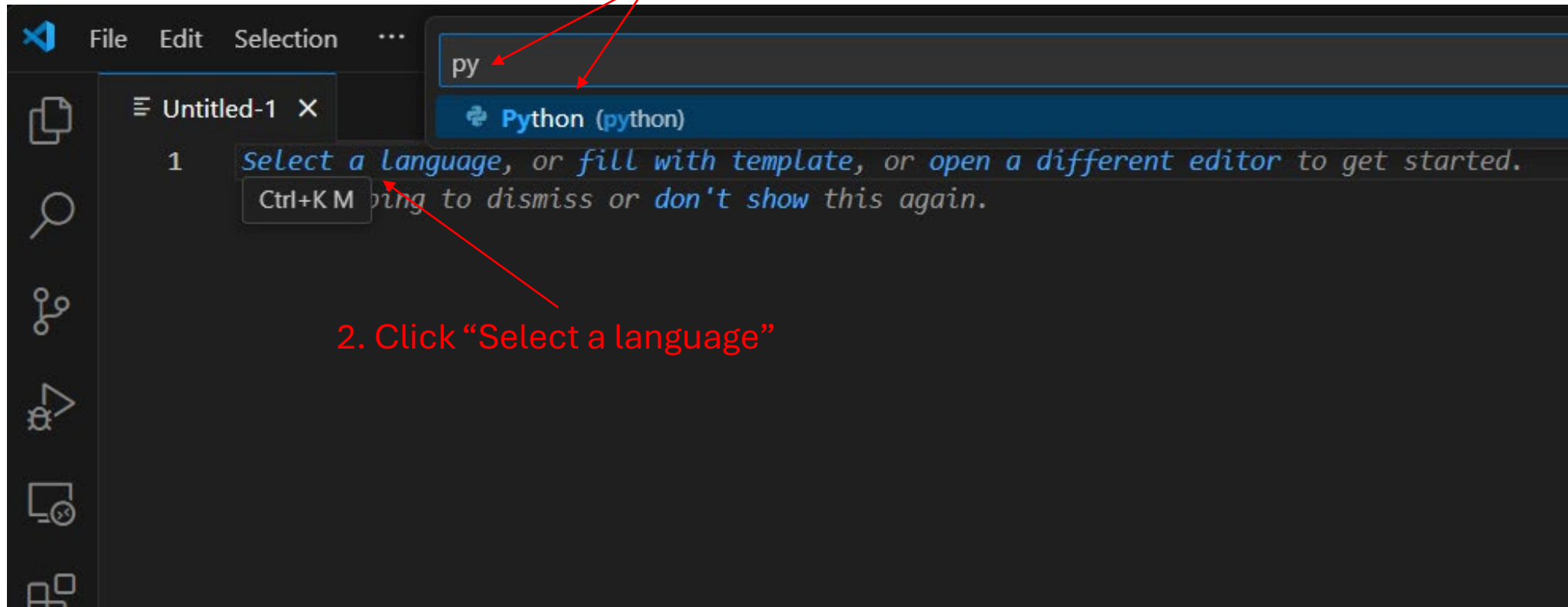
2. Click Install

3. Click the
Extensions icon in
the side bar again to
close this window

Create a New Python (.py) File

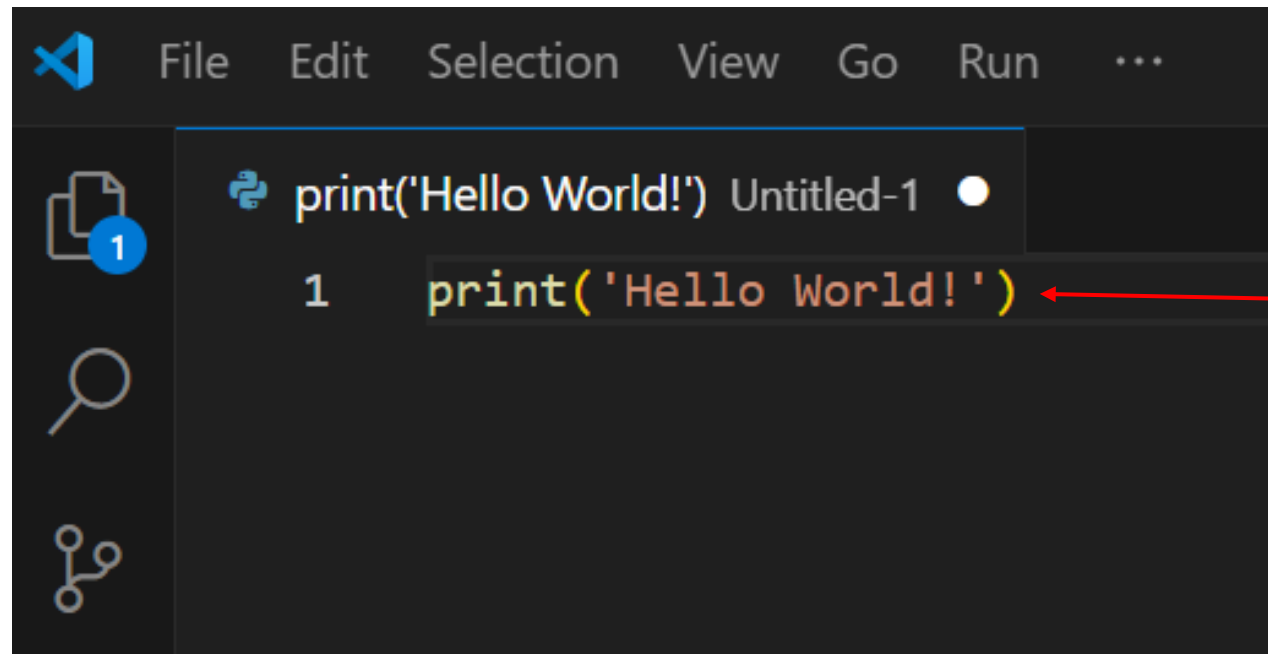
1. File > New Text File

3. Type “py” into the bar that opens, then click “Python”



2. Click “Select a language”

Write the traditional first line of code!



The screenshot shows the Visual Studio Code (VS Code) interface. The top menu bar includes 'File', 'Edit', 'Selection', 'View', 'Go', 'Run', and a dropdown menu. The left sidebar shows the Explorer view with a file icon and a search icon. The main editor area displays a Python file named 'Untitled-1'. The code 'print('Hello World!')' is written on line 1. A red arrow points to the code.

1.

2. Save the file
somewhere, e.g. as
test.py in your
Documents

3. Restart VSCode and
re-open this file

Link VS Code to Python

Note: If you don't see this yellow button, instead press:

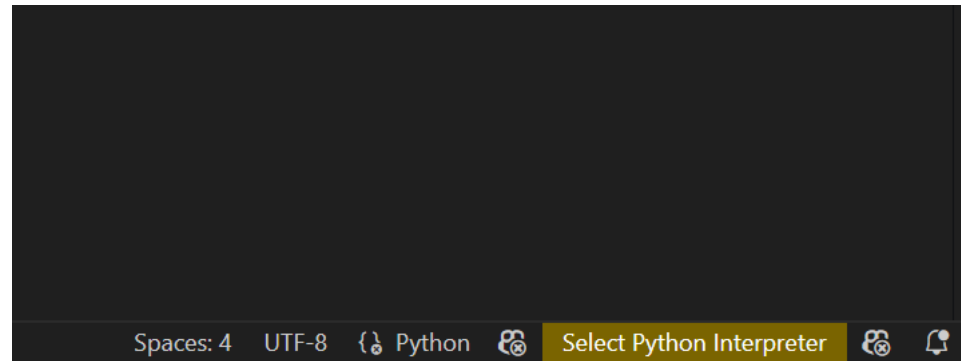


Ctrl + shift + p



Cmd + shift + p

Then type “Select: Py” and click on Python: Select Interpreter



Notice this appear in the bottom-right corner

Link VS Code to Python

Note: If you don't see this yellow button, instead press:



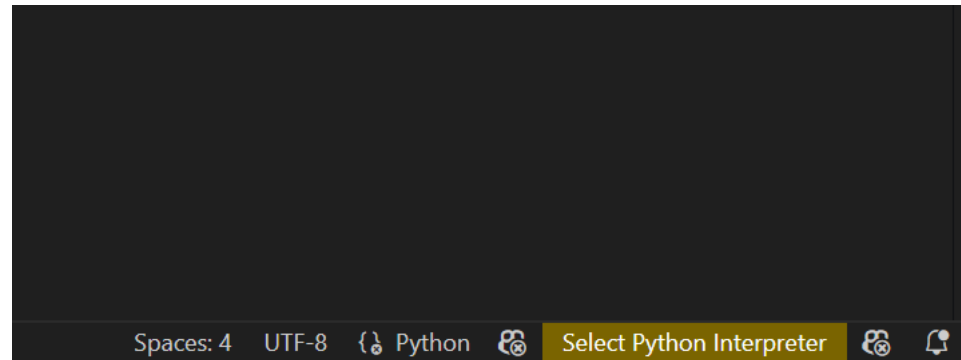
Ctrl + shift + p



Cmd + shift + p

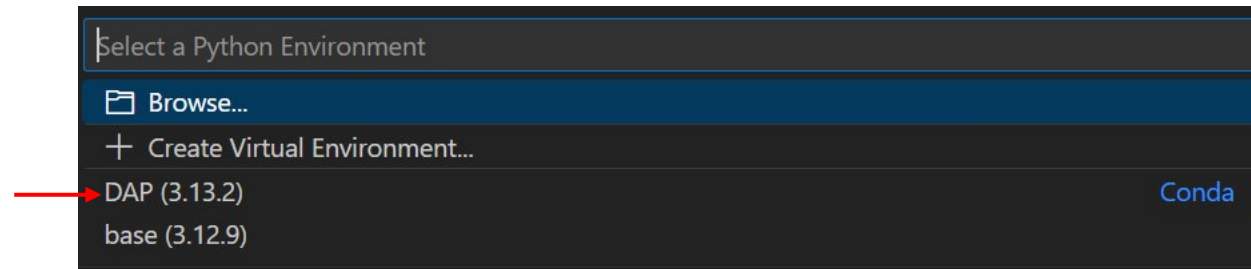
Then type “Select: Py” and click on Python: Select Interpreter

Note: The + in these commands does not literally mean to press the + key; it means to press “ctrl” and then press “shift” and then press “p”! We use this syntax throughout.



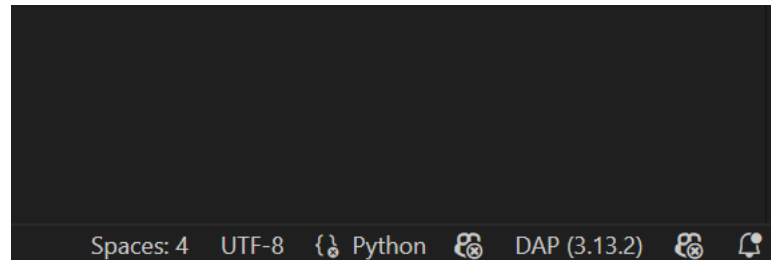
Notice this appear in the bottom-right corner

Link VS Code to Python



1. Click “Select Interpreter” from the previous slide
2. This menu will open at the top
3. Select “DAP”
4. Restart VSCode again
 - It may install some VSCode libraries

Link VS Code to Python



The “Select Interpreter” on the bottom-right now changes to show your selected interpreter

If it does not right away, continue to the next steps and it should resolve itself

Working with Text and Interpreters

- VSCode is highly customizable, and you should feel free to experiment!
- <https://code.visualstudio.com/docs/languages/python>
- For this class, however, we recommend you use the interactive interpreter

Interactive Interpreter in VSCode

First, enter the
Command Pallet:



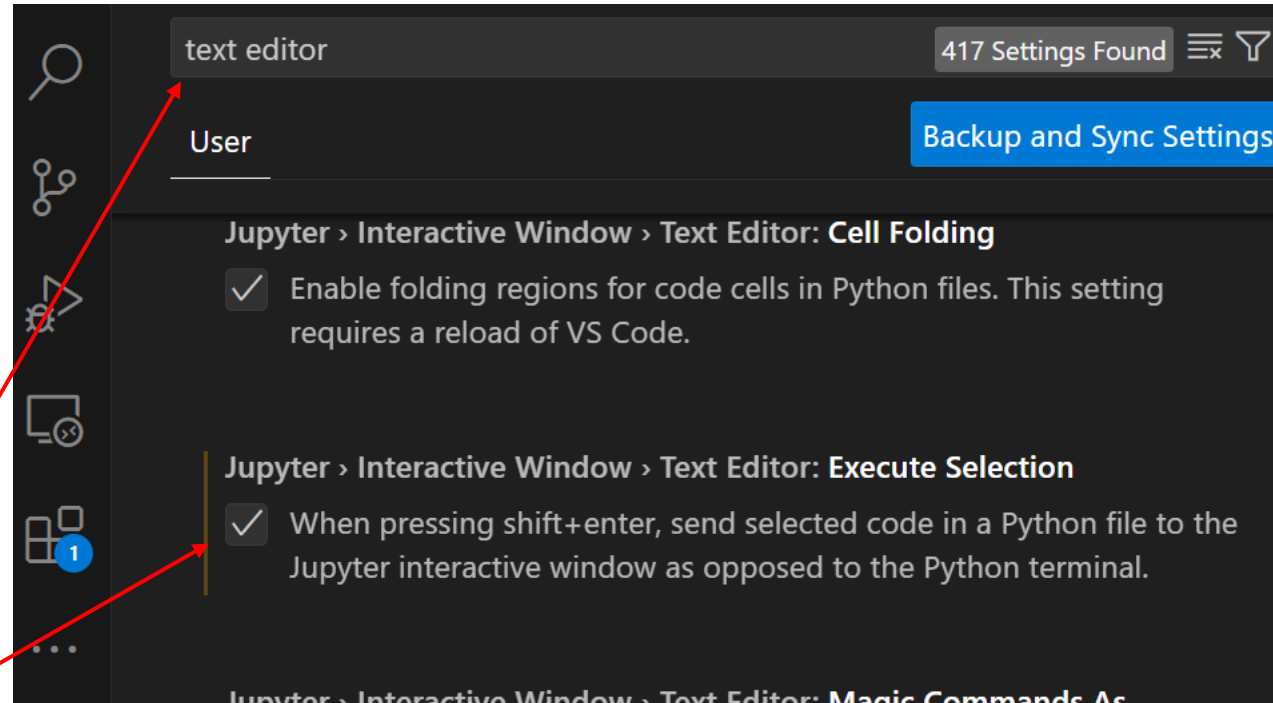
Ctrl + shift + p



Cmd + shift + p

Second, search for
“text editor”

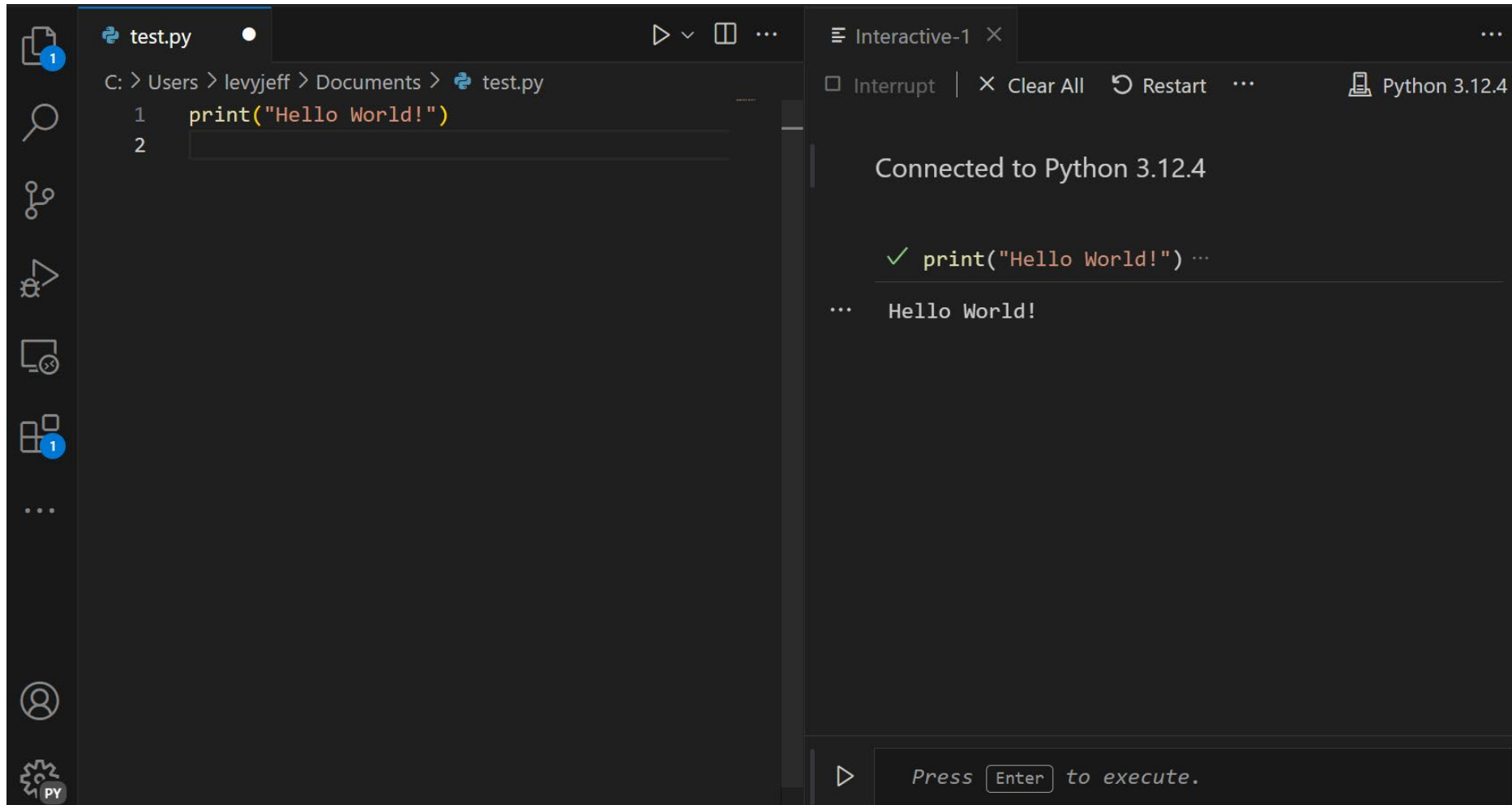
Third, check this box
as shown



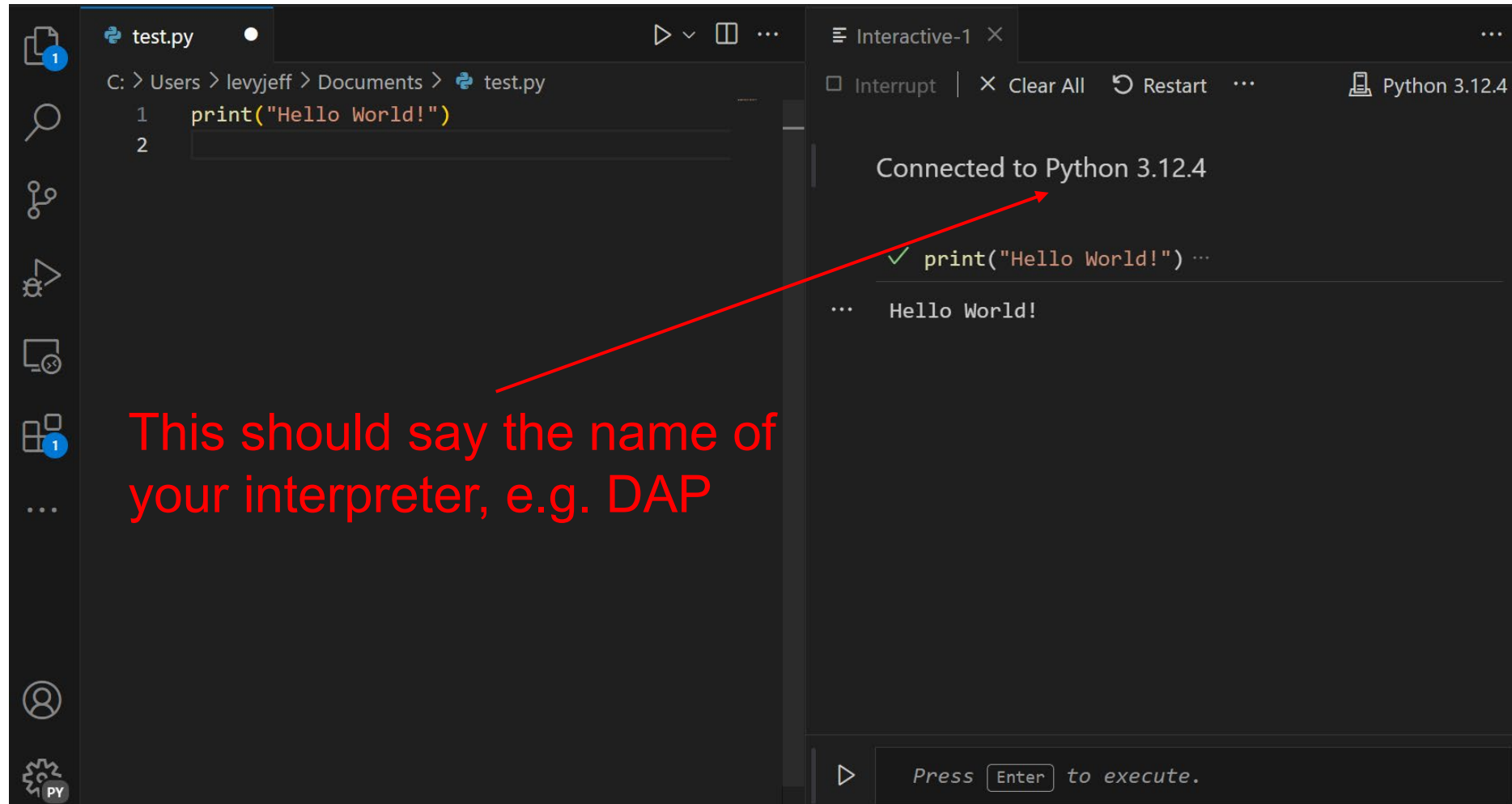
Interactive Interpreter in VSCode

1. Close the settings window and return to your test.py file
2. Place the cursor at the end of the print statement you wrote earlier
3. Press “shift + enter”
 - You will always execute code by selecting it, then pressing “shift + enter”

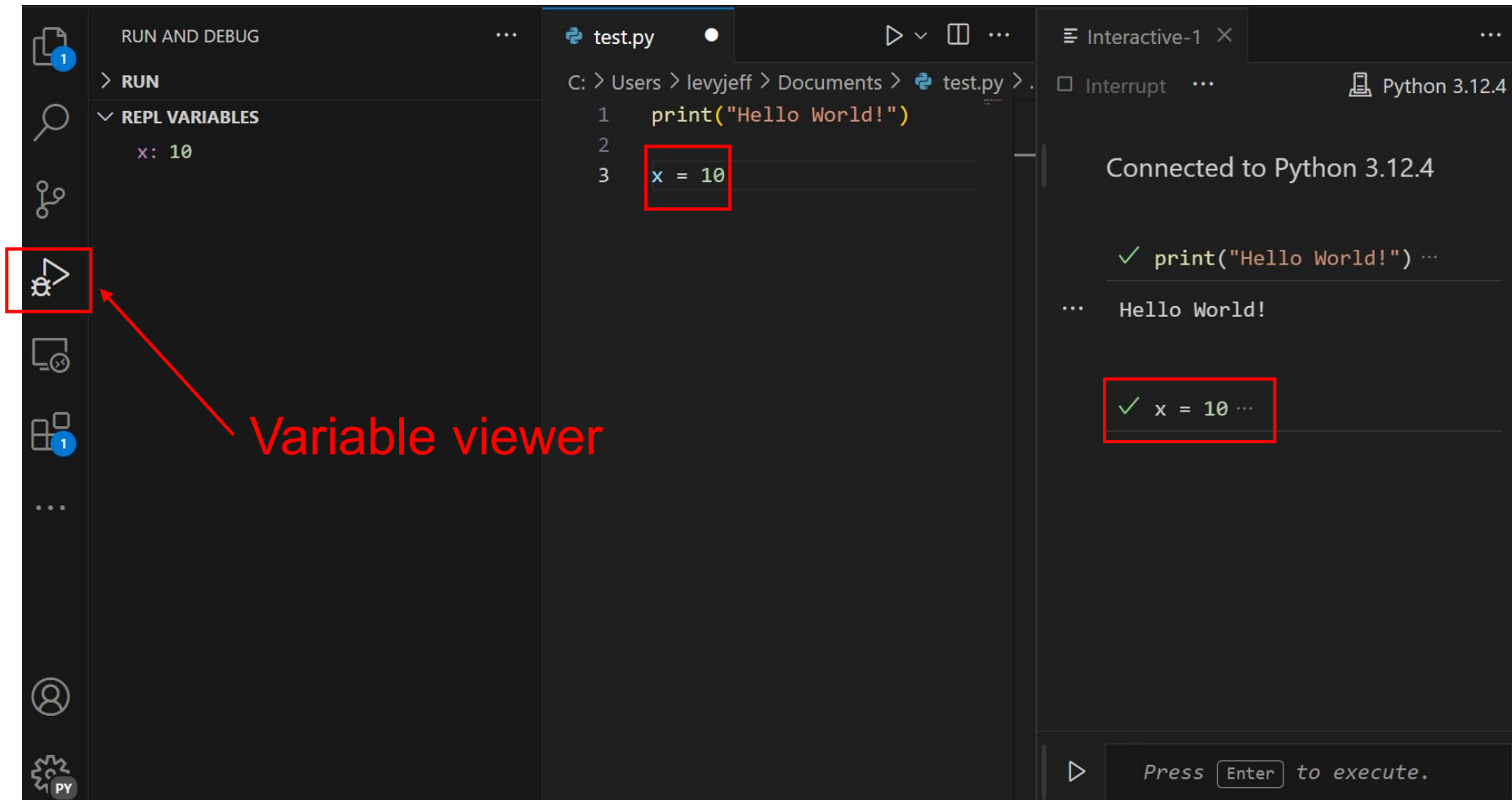
Interactive Interpreter in VSCode



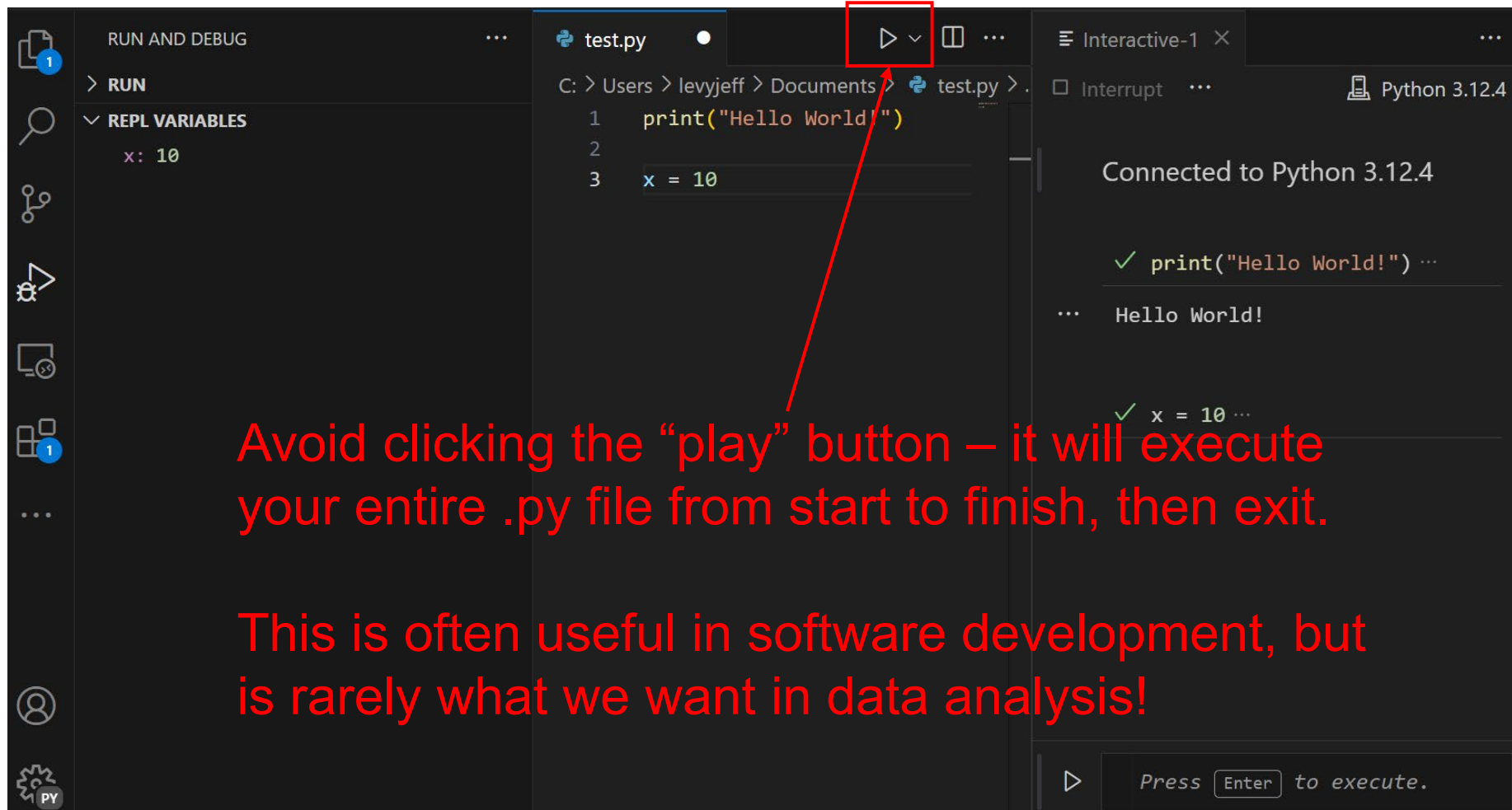
Interactive Interpreter in VSCode



3. Use Jupyter in VSCode



3. Use Jupyter in VSCode



Finish!

You are now set up to use Python!

Note that there are many other IDEs and text editors (to name just a few: Jupyter Notebooks, PyCharm, Spyder, Sublime, Vim).

You may eventually change, though again, it will be much easier for us to help you if you stick with the recommended setup in this class.

It also isn't unusual to use different editors for different purposes. For example, you might use a Jupyter Notebook for data exploration, and VSCode for project development.

Note that nearly all of these options (including VSCode) also work with R. This is also true in the other direction, however! Rstudio can work with Python.